

Python Tip: `is` vs. `==`

These operators may look similar, but they answer different questions.

`==` compares values.

`is` compares identity.



Using ==

== checks whether two objects have the same value.

```
first = ["python", "sql"]
second = ["python", "sql"]

print("Lists are equal:", first == second)

name = "Ada"
print("Name equals 'Ada':", name == "Ada")
```

```
Lists are equal: True
Name equals 'Ada': True
```

Using `is`

`is` checks whether two names point to the **same object**.

```
first = ["python", "sql"]
second = first

print("Names point to the same list:", first is second)

# Changing one changes both because they are the same object!
second.append("git")
print("First list after update:", first)
```

```
Names point to the same list: True
First list after update: ['python', 'sql', 'git']
```

Both names refer to one list, so a change through either name affects it.

Equal, but Not Identical

Two separate objects can contain equal values.

```
original = ["python", "sql"]  
other = ["python", "sql"]  
  
print("Lists are equal:", original == other)  
print("Lists are same:", original is other)
```

```
Lists are equal: True  
Lists are same: False
```

Use `==` when you care about what the objects contain.

Checking for None

Objects can customize how `==` behaves. Identity cannot be overridden.

```
class TrickyObject:
    def __eq__(self, other):
        return True

not_none = TrickyObject()
yes_none = None

print("Custom object == None:", not_none == None)
print("Custom object is None:", not_none is None)
print("None == None:", yes_none == None)
print("None is None:", yes_none is None)
```

```
Custom object == None: True
Custom object is None: False
None == None: True
None is None: True
```

Use `is None` to reliably check for the one and only `None` object.

An Apparent Exception

Sometimes Python treats small integers as the same object for optimization. Don't be fooled into thinking this applies to all integers!

```
x1 = 10
x2 = 9 + 1  # x1 == x2
y1 = 9999999
y2 = 9999998 + 1  # y1 == y2

print("Small ints are same object:", x1 is x2)
print("Big ints are same object:", y1 is y2)
```

```
Small ints are same object: True
Big ints are same object: False
```

This may make `is` appear to work as equality for small integers—but do not rely on it! It is an implementation detail, not the rule. Always use `==` to compare numeric values.

Wrap-Up

Use `==` to ask: **Do these have the same value?**

Use `is` to ask: **Are these the exact same object?**

Most comparisons need `==`. Use `is` when you need to check for identity, such as when checking for `None`.

Follow me for more tips.

Shep Bryan IV